COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/16/2026	**DATE SUBMITTED:**	9/16/2026
NAME:	Katindig, Jiro Luis	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 **Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot	HOA > main.cpp > main()
	`#include <iostream>`
	`#include <cstdlib>`
	`#include <ctime>`
	`#include "sortingAlgo2.h"`
	
	`const int arrSize = 100;`
	`void displayArr(int Arr[]);`
	
	`int main()`
	`{`
	`  // seed the random number generator once for the whole program`
	`  srand(time(0));`
	
	`  // Array 1: Shell Sort`
	`  int shellArr[arrSize];`
	`  for (int i = 0; i < arrSize; i++)`
	`  {`
	`  shellArr[i] = rand() % 1000;`
	`  }`
	`  std::cout << "Array 1 - Unsorted:\n";`
	`  displayArr(shellArr);`
	
	Array 1 - Unsorted:
	914 180 22 171 937 687 766 649 701 255
	190 201 741 89 322 949 368 57 528 436
	40 925 434 712 420 477 650 124 734 950
	509 398 525 616 11 937 939 37 319 238
	425 675 90 796 730 67 148 781 254 554
	901 347 256 240 977 595 865 400 139 216
	438 391 283 539 952 1 954 674 675 96
	950 895 961 686 913 851 336 341 31 291
	633 916 576 825 586 749 778 789 718 141
	803 262 611 820 365 83 332 350 763 463

Observations	The program generates 100 random integers and prints them in rows of 10. This is the base unsorted data set that all three sorting algorithms will use for testing.

Table 8-1. Array of Values for Sort Algorithm Testing

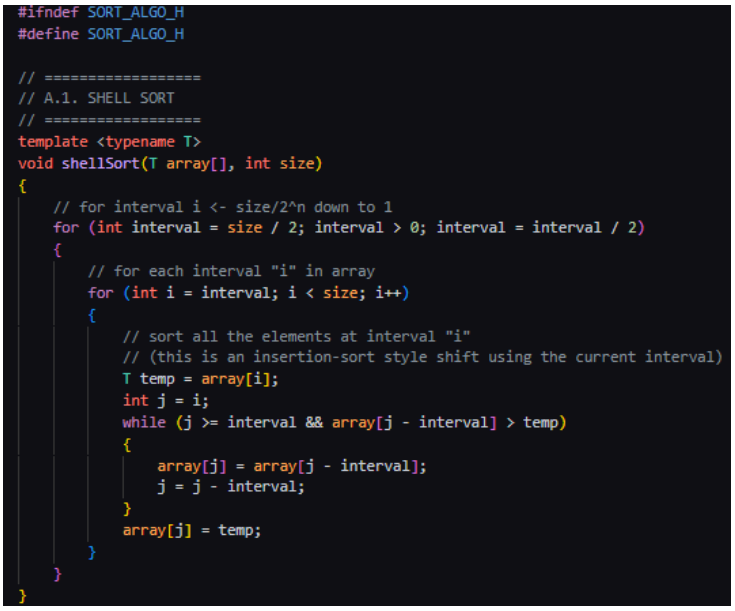
Code + Console Screens hot

```
// Array 1: Shell Sort
int shellArr[arrSize];
for (int i = 0; i < arrSize; i++)
{
    shellArr[i] = rand() % 1000;
}
std::cout << "Array 1 - Unsorted:\n";
displayArr(shellArr);

shellSort(shellArr, arrSize);
std::cout << "\nArray 1 - Sorted using Shell Sort:\n";
displayArr(shellArr);
```

```
Array 1 - Unsorted:
914 180 22 171 937 687 766 649 701 255
190 201 741 89 322 949 368 57 528 436
40 925 434 712 420 477 650 124 734 950
509 398 525 616 11 937 939 37 319 238
425 675 90 796 730 67 148 781 254 554
901 347 256 240 977 595 865 400 139 216
438 391 283 539 952 1 954 674 675 96
950 895 961 686 913 851 336 341 31 291
633 916 576 825 586 749 778 789 718 141
803 262 611 820 365 83 332 350 763 463

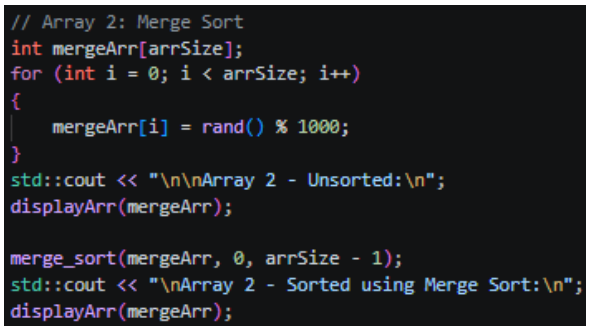
Array 1 - Sorted using Shell Sort:
1 11 22 31 37 40 57 67 83 89
90 96 124 139 141 148 171 180 190 201
216 238 240 254 255 256 262 283 291 319
322 332 336 341 347 350 365 368 391 398
400 420 425 434 436 438 463 477 509 525
528 539 554 576 586 595 611 616 633 649
650 674 675 675 686 687 701 712 718 730
734 741 749 763 766 778 781 789 796 803
820 825 851 865 895 901 913 914 916 925
937 937 939 949 950 950 952 954 961 977
```

	
Observations	Shell Sort compares elements that are far apart first using a shrinking gap, then narrows the gap down until it becomes a regular insertion sort. The output confirms the array is correctly sorted from smallest to largest.

```
#ifndef SORT_ALGO_H
#define SORT_ALGO_H

// =================
// A.1. SHELL SORT
// =================
template <typename T>
void shellSort(T array[], int size)
{
    // for interval i <- size/2^n down to 1
    for (int interval = size / 2; interval > 0; interval = interval / 2)
    {
        // for each interval "i" in array
        for (int i = interval; i < size; i++)
        {
            // sort all the elements at interval "i"
            // (this is an insertion-sort style shift using the current interval)
            T temp = array[i];
            int j = i;
            while (j >= interval && array[j - interval] > temp)
            {
                array[j] = array[j - interval];
                j = j - interval;
            }
            array[j] = temp;
        }
    }
}
```

Table 8-2. Shell Sort Technique

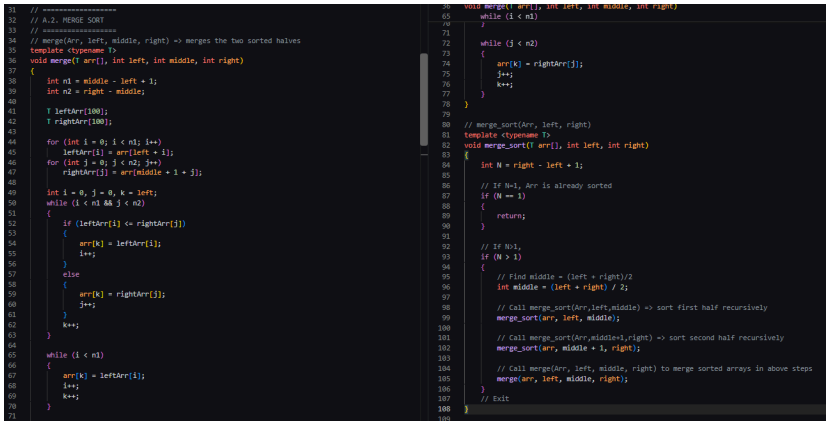
Code + Console Screenshot	

```
// Array 2: Merge Sort
int mergeArr[arrSize];
for (int i = 0; i < arrSize; i++)
{
    mergeArr[i] = rand() % 1000;
}
std::cout << "\n\nArray 2 - Unsorted:\n";
displayArr(mergeArr);

merge_sort(mergeArr, 0, arrSize - 1);
std::cout << "\nArray 2 - Sorted using Merge Sort:\n";
displayArr(mergeArr);
```

```
Array 2 - Unsorted:
97 113 411 629 172 584 805 200 413 230
58 103 825 143 280 646 517 108 990 728
873 597 379 708 563 653 164 926 634 648
445 847 533 920 791 527 290 632 859 906
267 16 494 153 982 203 998 587 755 629
940 388 364 864 162 932 360 425 31 506
960 615 961 531 831 4 122 454 147 832
270 426 953 257 475 922 88 335 970 48
755 439 597 519 53 382 938 237 513 25
423 113 712 243 598 378 751 207 843 585

Array 2 - Sorted using Merge Sort:
4 16 25 31 48 53 58 88 97 103
108 113 113 122 143 147 153 162 164 172
200 203 207 230 237 243 257 267 270 280
290 335 360 364 378 379 382 388 411 413
423 425 426 439 445 454 475 494 506 513
517 519 527 531 533 563 584 585 587 597
597 598 615 629 629 632 634 646 648 653
708 712 728 751 755 755 791 805 825 831
832 843 847 859 864 873 906 920 922 926
932 938 940 953 960 961 970 982 990 998
```



```
// ==================
// A.2. MERGE SORT
// ==================
// merge(Arr, left, middle, right) => merges the two sorted halves
template <typename T>
void merge(T arr[], int left, int middle, int right)
{
    int n1 = middle - left + 1;
    int n2 = right - middle;

    T leftArr[100];
    T rightArr[100];

    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[middle + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2)
    {
        if (leftArr[i] <= rightArr[j])
        {
            arr[k] = leftArr[i];
            i++;
        }
        else
        {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }

    while (i < n1)
    {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
}

// merge_sort(Arr, left, right)
template <typename T>
void merge_sort(T arr[], int left, int right)
{
    int N = right - left + 1;

    // If N=1, Arr is already sorted
    if (N == 1)
    {
        return;
    }

    // If N>1,
    if (N > 1)
    {
        // Find middle = (left + right)/2
        int middle = (left + right) / 2;

        // Call merge_sort(Arr,left,middle) => sort first half recursively
        merge_sort(arr, left, middle);

        // Call merge_sort(Arr,middle+1,right) => sort second half recursively
        merge_sort(arr, middle + 1, right);

        // Call merge(Arr, left, middle, right) to merge sorted arrays in above steps
        merge(arr, left, middle, right);
    }
    // Exit
}
```

Observations	Merge Sort keeps splitting the array into halves until single elements remain, then merges them back together in order. It reliably produces a fully sorted array, though it needs extra array space to hold the merged halves.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```
// Array 3: Quick Sort
int quickArr[arrSize];
for (int i = 0; i < arrSize; i++)
{
    quickArr[i] = rand() % 1000;
}
std::cout << "\n\nArray 3 - Unsorted:\n";
displayArr(quickArr);

quicksort(quickArr, 0, arrSize - 1);
std::cout << "\nArray 3 - Sorted using Quick Sort:\n";
displayArr(quickArr);

return 0;
```

```
Array 3 - Unsorted:
103 686 179 911 206 938 312 201 727 284
995 891 117 92 337 491 102 487 744 624
982 997 203 43 353 491 133 484 805 487
499 176 654 203 61 441 212 627 958 327
547 458 593 142 730 722 175 911 806 786
718 252 711 469 180 670 482 825 901 801
185 157 809 259 361 359 642 25 906 12
236 714 28 609 613 984 624 818 999 236
524 929 180 977 92 689 730 684 997 907
727 50 299 185 902 411 150 633 674 598

Array 3 - Sorted using Quick Sort:
12 25 28 43 50 61 92 92 102 103
117 133 142 150 157 175 176 179 180 180
185 185 201 203 203 206 212 236 236 252
259 284 299 312 327 337 353 359 361 411
441 458 469 482 484 487 487 491 491 499
524 547 593 598 609 613 624 624 627 633
642 654 670 674 684 686 689 711 714 718
722 727 727 730 730 744 786 801 805 806
809 818 825 891 901 902 906 907 911 911
929 938 958 977 982 984 995 997 997 999
```

```cpp
// ==================
// A.3. QUICK SORT
// ==================
// partition(A, low, high)
template <typename T>
int partition(T A[], int low, int high)
{
    T pivotValue = A[high]; // last element as pivot
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (A[j] < pivotValue)
        {
            i++;
            T temp = A[i];
            A[i] = A[j];
            A[j] = temp;
        }
    }

    T temp = A[i + 1];
    A[i + 1] = A[high];
    A[high] = temp;

    return i + 1;
}

// quicksort(A, low, high)
template <typename T>
void quicksort(T A[], int low, int high)
{
    // if(low < high)
    if (low < high)
    {
        // pivot = partition(A,low,high);
        int pivot = partition(A, low, high);

        // quicksort(A,low,pivot-1)
        quicksort(A, low, pivot - 1);

        // quicksort(A,pivot+1,high)
        quicksort(A, pivot + 1, high);
    }
}

#endif
```

Observations	Quick Sort picks a pivot, moves smaller values to its left and larger values to its right, then repeats this on each side. It sorted the array correctly and generally ran fast since it sorts in place without needing extra arrays.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Yes, we can. After partition() splits the array into a left sublist (smaller than the pivot) and a right sublist (larger than the pivot), those sublists are just regular arrays, so any sorting algorithm can sort them

instead of calling quicksort() again. For example, with the array {12, 8, 3, 15, 20} and pivot = 12, we get left sublist {8, 3} and right sublist {15, 20}. Instead of recursively calling quicksort() on {8, 3}, we could use shellSort() on it instead, and it would still correctly sort it into {3, 8}.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

I think Quick Sort would be the fastest for this array since it is a small list and Quick Sort does not need extra arrays like Merge Sort does, so it has less overhead. As for why Merge Sort and Quick Sort are both O(N log N), it is because they both work by splitting the array in half over and over. We can only cut an array in half so many times before we are down to single elements, and that number of times is log N. At each of those levels, the algorithm still has to go through all N elements to merge them back together or partition them around a pivot. So we end up multiplying the log N levels by the N work per level, which gives us N log N overall.

CONCLUSION:

This activity taught me how different sorting algorithms actually work once I saw their code and output side by side. I learned that Shell Sort improves on simple insertion sort by comparing far apart elements first, Merge Sort splits the array and merges it back together, and Quick Sort partitions the array around a pivot. Seeing all three run on the same data helped me understand their differences instead of just memorizing their steps.

I also learned a lot about how these algorithms perform in terms of speed and memory use. Merge Sort and Quick Sort are both much faster than simpler methods because they use a divide and conquer approach, which is why they reach O(N log N) time complexity. Overall, this activity helped me understand sorting algorithms on a deeper level and gave me a better appreciation of why choosing the right algorithm matters.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.